

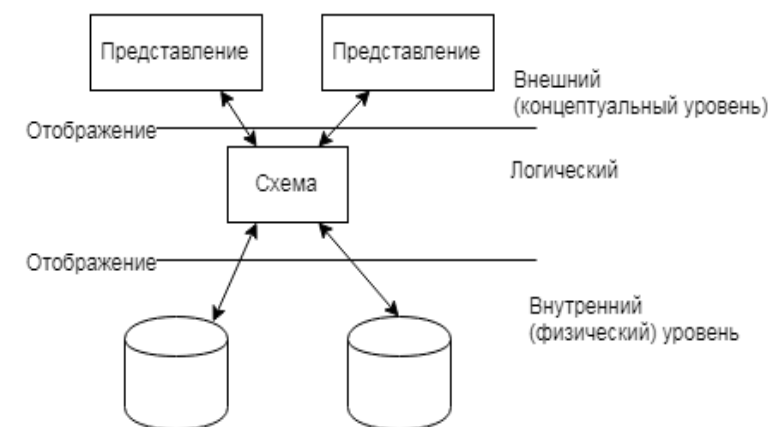
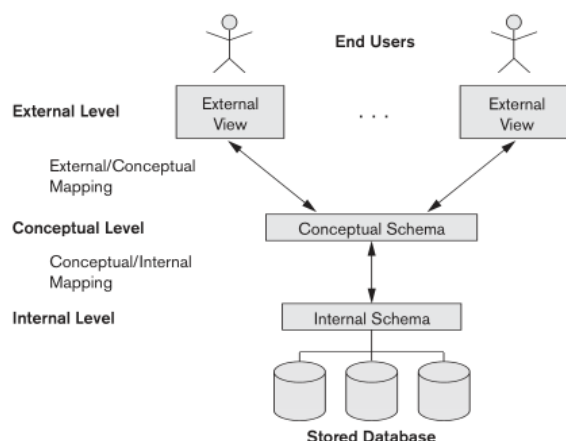
Оглавление

1	Логическая архитектура СУБД.....	2
2	Модели данных.....	6
2.1	Понятие модели данных.....	6
2.2	Составляющие модели данных	7
2.3	Классификация моделей данных.....	7
3	Концептуальное представление данных	9
3.1	Концептуальное (семантическое) моделирование данных.....	9
3.2	Сущность и ее свойства.....	9
3.2.1	Понятие сущности.....	9
3.2.2	Ключи	10
3.3	Связи концептуальных объектов.....	10
3.3.1	Наследование и иерархическая категоризация	10
3.3.2	Агрегирование	12
3.3.3	Ссылка	13
3.3.4	Множественная ассоциация	14
3.4	Порядок инфологического (семантического) проектирования БД.....	15
3.4.1	Разработка семантической/концептуальной модели	15
3.4.2	Верификация семантической/концептуальной модели.....	15
3.5	Описание ограничений целостности.....	16
3.5.1	Виды целостности (обеспечения целостности) в СУБД	16
3.5.2	Понятие ограничения целостности (с точки зрения ссылок и семантики)	17
3.5.3	Когда проводить проверку	17
3.5.4	Классификация ограничений целостности	17

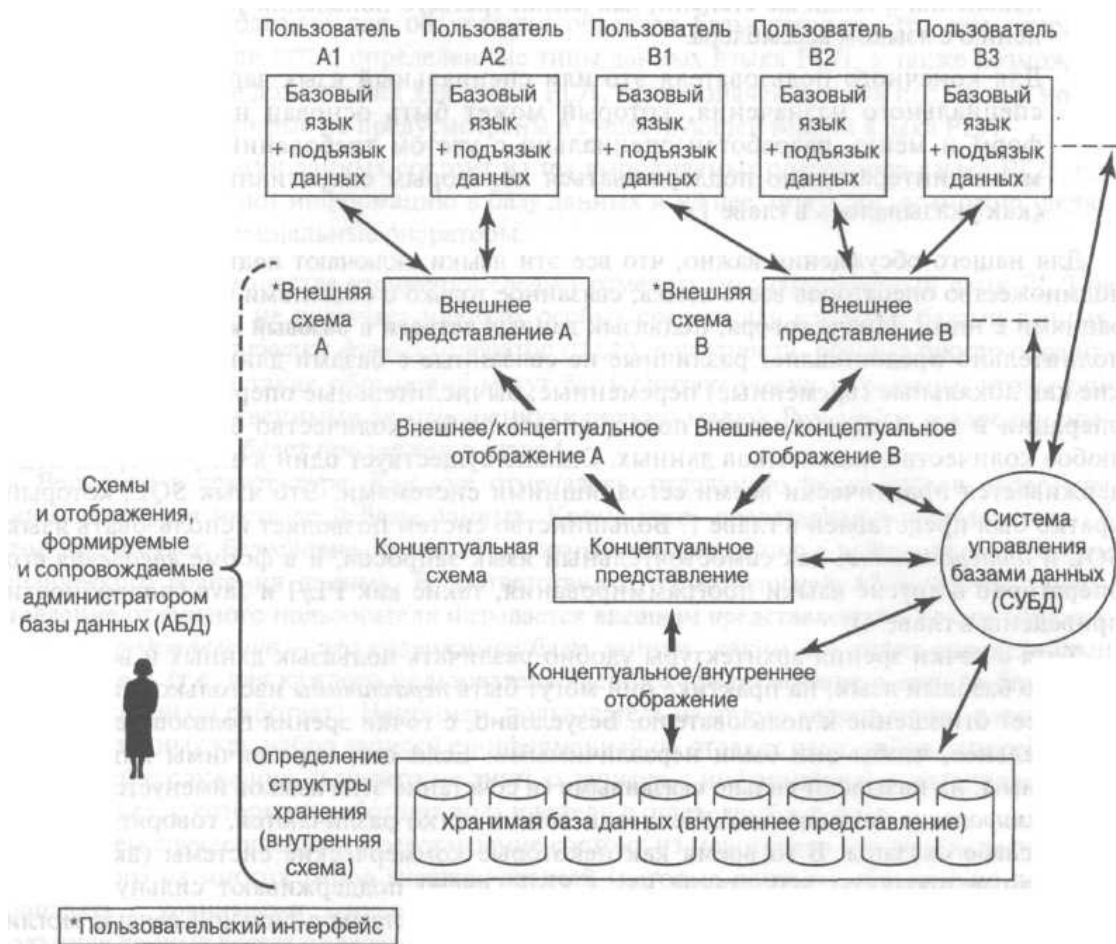
1 ЛОГИЧЕСКАЯ АРХИТЕКТУРА СУБД

ANSI/SPARC

Научная группа ANSI/SPARC (Study Group on Data Base Management Systems) была организована в 1972 комитетом Standards Planning and Requirements Committee (SPARC) института American National Standards Institute on Computers and Information Processing (ANSI/X3). • Целью группы являлось определение областей технологии баз данных, нуждающихся в стандартизации, и выработка набора рекомендуемых действий в каждой из этих областей. • В 1975-78 группа разработала архитектуру системы баз данных



Внешний (PL/I)	Внешний (COBOL)
DCL 1 EMP#, 2 EMP# CHAR(6), 2 SAL FIXED BIN(31);	01 EMP#. 02 EMPNO PIC X(6). 02 DEPTNO PIC X(4).
Концептуальный	
EMPLOYEE EMPLOYEE_NUMBER CHARACTER(6) DEPARTMENT_NUMBER CHARACTER(4) SALARY DECIMAL(5)	
Внутренний	
STORED_EMP BYTES=20 PREFIX BYTES=6,OFFSET=0 EMP# BYTES=6,OFFSET=6,INDEX=EMPX DEPT# BYTES=4,OFFSET=12 PAY BYTES=4,ALIGN=FULLWORD,OFFSET=16	



Трехсхемная (трехуровневая) архитектура:

1. Использование каталога хранилища СУБД для хранения описания базы данных (схемы)
2. Изоляция программ и данных и операционная независимость за счет внешних схем
3. Поддержка многопользовательского режима (за счет внешних схем)

Физический уровень (The internal level) – физ. Модель и физ. Представление/манипулирование данными.

Внутреннее представление, так же как внешнее и концептуальное, отделено от физического уровня, поскольку в нем **не рассматриваются физические записи, обычно называемые блоками или страницами, и физические области устройства хранения, такие как цилиндры и дорожки**. Другими словами, внутреннее представление предполагает наличие бесконечного **линейного адресного пространства**.

Внутреннее представление **описывается с помощью внутренней схемы**, которая определяет не только различные типы хранимых записей, но также существующие индексы, способы представления хранимых полей, физическую упорядоченность хранимых записей и т.д

Логический уровень (The conceptual level) – содержит концептуальное описание данных, часто в виде внутренней (промежуточной) схемы. Эта концептуальная схема реализации часто основана на концептуальной схеме проектирования в высокоуровневой модели данных.

Концептуальный уровень (The external or view level) – пользовательские представления.

Граница между уровнями может быть не четкой. Например, концептуальный уровень может содержать элементы физического представления или отсутствовать в выраженном виде (NoSQL).

Некоторые СУБД позволяют использовать различные модели данных на концептуальном и внешнем уровнях (например, у IBM используется реляционная модель на внутреннем уровне и

объектно – ориентированная в пользовательских представлениях). С точки зрения проектирования так бывает практически всегда.

Подобная архитектура позволяет соблюсти **независимость данных**.

Logical data independence – возможность изменить концептуальную (внутреннюю) схему без изменения внешних представлений или приложений. *Only the view definition and the mappings need to be changed in a DBM*

Physical data independence – возможность изменять физическую схему без изменения логической (посмотрим на примере миграции, UPD баз данных)

На каждом уровне используется та или иная *модель данных*.

Модели данных (Data models) – набор концепций (абстракция) описывающая структуру базы данных.¹

Виды моделей данных с точки зрения архитектуры:

- высокоуровневые (концептуальные) (сущность – связь)
- representational (or implementation) – логические. Репрезентативные модели данных скрывают многие детали хранения данных на диске, но могут быть реализованы непосредственно в компьютерной системе. (Реляционная, объектная для ряда систем)
- низкоуровневые (физические) информация, такая как форматы записей, порядок записей и путь доступа

Схемы (Schemas) – описание базы данных (самой себя) внутри БД. (часто представляются в диаграммах) – в ряде серверов могут быть собственные представления понятию «схема» (напр. SQL Server). Исторические изменения в схеме называют ее эволюцией.

Экземпляры (Instances) – снимок базы данных в конкретный момент времени (включая все хранимые данные и их состояние на этот момент).

Состояния базы данных:

- Непротиворечивость (не взаимозаменяемы)
- Истинность (всегда непротиворечива)

Метаданные – данные в СУБД, описывающие ее схему и составляющие (данные о данных).

Языки базы данных

Архитектурные языки:

data definition language (DDL) – используется для определения данных (работы со схемой)

storage definition language (SDL) – используется для спецификации внутренней схемы (*In most relational DBMSs today, there is no specific language that performs the role of SDL*) Физическая схема определяется набором функций, параметров и спецификаций связанных с хранилищем.

¹ Elmasri, Ramez. Fundamentals of database systems / Ramez Elmasri, Shamkant B. Navathe. — 6th ed. p. cm. Includes bibliographical references and index. ISBN-13: 978-0-136-08620-8

view definition language (VDL) – используется для определения представлений и отображений пользовательского уровня. (*the DDL is used to define both conceptual and external schemas*)

data manipulation language (DML) – манипулирование данными

Для всех видов используется один интегрированный язык SQL (DDL, VDL, and DML – SDL был в ранних версиях но потом удален). В не реляционных СУБД ситуация может отличаться.

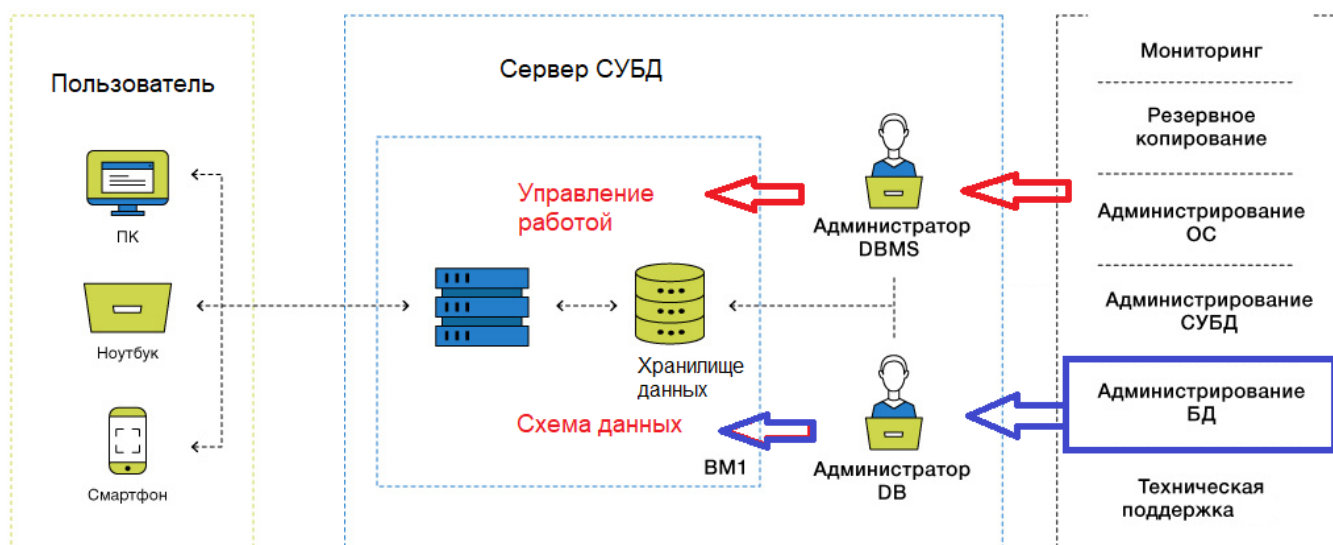
2 вида DML

A high-level or nonprocedural DML – описывает комплексные операции над базой данных. Может быть включен в интегрированный язык или отсутствовать.

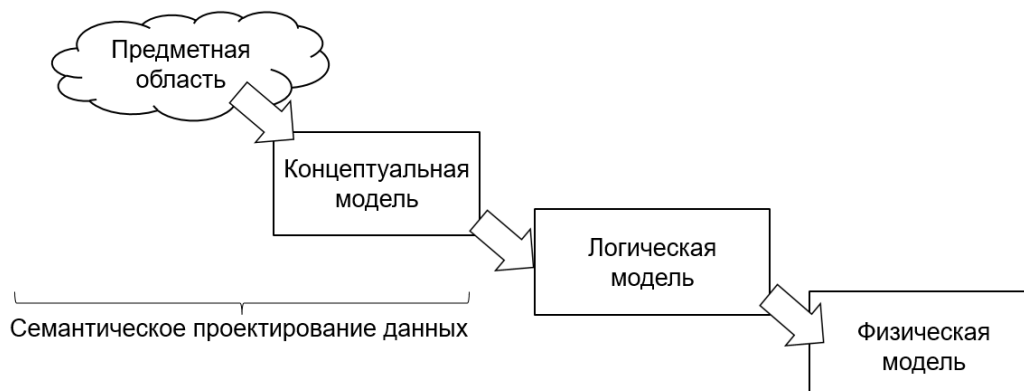
A low-level or procedural DML – должен быть включен в интегрированный язык – включает манипуляцию индивидуальными объектами, записями, значениями и т.д.

Функции администрирования базы данных

1. Определение концептуальной схемы
2. Определение внутренней схемы
3. Взаимодействие с пользователями
4. Определение требований защиты и обеспечения целостности данных
5. Определение процедур резервного копирования и восстановления
6. Управление производительностью и реагирование на изменяющиеся требования



Порядок проектирования БД:



2 МОДЕЛИ ДАННЫХ

2.1 Понятие модели данных

В современной его трактовке термин "модель данных" и обозначает *инструмент моделирования*, а вовсе не результат, в то время как «модель базы данных» (представляющая собой схему базы данных) или «модель предметной области» (спецификации модели предметной области) являются *результатами моделирования*.

Модель данных является ядром любой базы данных, она определяет правила формализованного описания объектов предметной области и взаимосвязей между ними. Для этого она фиксирует допустимые структуры данных, соглашения о способах их представления и возможные операции манипулирования ими. Поэтому, естественными требованиями к модели данных являются:

1. *Модель должна быть достаточно универсальной*, позволяя работать с данными различной структуры и сложности.
2. *Модель должна допускать автоматическую обработку данных*, т.е. должна быть реализуема программными средствами.
3. *Модель должна быть наглядной, «прозрачной»*. Поскольку задача описания структуры данных средствами выбранной модели возлагается на разработчика (человека), чем сложнее модель - тем труднее избежать ошибок при проектировании.

Модели данных различаются, прежде всего, допустимыми структурами данных.

В программировании проблематика абстрактных типов данных основной акцент делает на

- а) создании общего инструмента конструирования типа данных
- б) на соответствующей технике верификации.

В моделях данных основной акцент делается на определении таких категорий типов данных, которые могли бы использоваться во многих ситуациях.

Сильно типизированные модели данных предполагают, что все данные отнесены к какой-либо категории (например, СЛУЖАЩИЙ). Свойства категорий наследуются принадлежащими к ним данными.

Слабо типизированные модели данных не связаны предположениями относительно категорий. Категории используются в той степени, в какой это целесообразно в каждом конкретном случае.

Первая модель данных, которая была разработана, была реляционная модель (1969г.). Бытует ошибочное мнение что иерархические и сетевые модели предшествовали реляционной модели. Это происходит из-за смешения понятий конкретных реализаций языков СУБД с одной стороны и моделей данных с другой. До 1970г были разработаны иерархические и сетевые системы, но модели данных для этих систем были определены только в 1973г.

Модели данных используются:

1. как инструмент для того, чтобы определять виды данных и их организацию, которые являются допустимыми в базах данных;
2. как основа разработки методологии создания для баз данных;

3. как основа для создания семейства языков высокого уровня манипулирования данными
4. как фокус разработки архитектуры СУБД;
5. как фактор обеспечения эволюции данных, с минимальным воздействием на прикладные программы.
6. как отправная точка для исследований по манипуляционным свойствам или альтернативным организациям данных.

2.2 Составляющие модели данных

Впервые термин "**модель данных**" был введен в 1981 году в статье Кодда под названием "Модели данных в управлении базами данных". Там **модель данных** определена как комбинация трех компонентов:

1. **Коллекции типов объектов данных**, образующих базовые строительные блоки для любой базы данных, соответствующей модели.
2. **Коллекции общих правил целостности (отношения)**, ограничивающих набор экземпляров тех типов объектов, которые законным образом могут появиться в любой такой базе данных.
3. **Коллекции общих правил целостности (ограничения)**, ограничивающих набор экземпляров тех типов объектов, которые законным образом могут появиться в любой такой базе данных.
4. **Коллекции операций**, применимых к таким экземплярам объектов для выборки и других целей.

В модели данных описывается некоторый набор родовых понятий и признаков, которыми должны обладать все конкретные СУБД и управляемые ими базы данных, если они основываются на этой модели.

Хотя понятие модели данных было введено Коддом, расширенная трактовка дана Дейтом с выделением в явном виде всех четырех составляющих

1. типы данных (объекты, таблицы и т.д.)
2. отношения между данными
3. ограничения, накладываемые на данные
4. (часто) операции, проводимые над данными

2.3 Классификация моделей данных

1.Инфологические - описывает смысловые содержания, здесь происходит выделение сущности объекта и связи между сущностями.

2.Даталогические - строятся на основе инфологических. Это модели для создания конкретной СУБД.

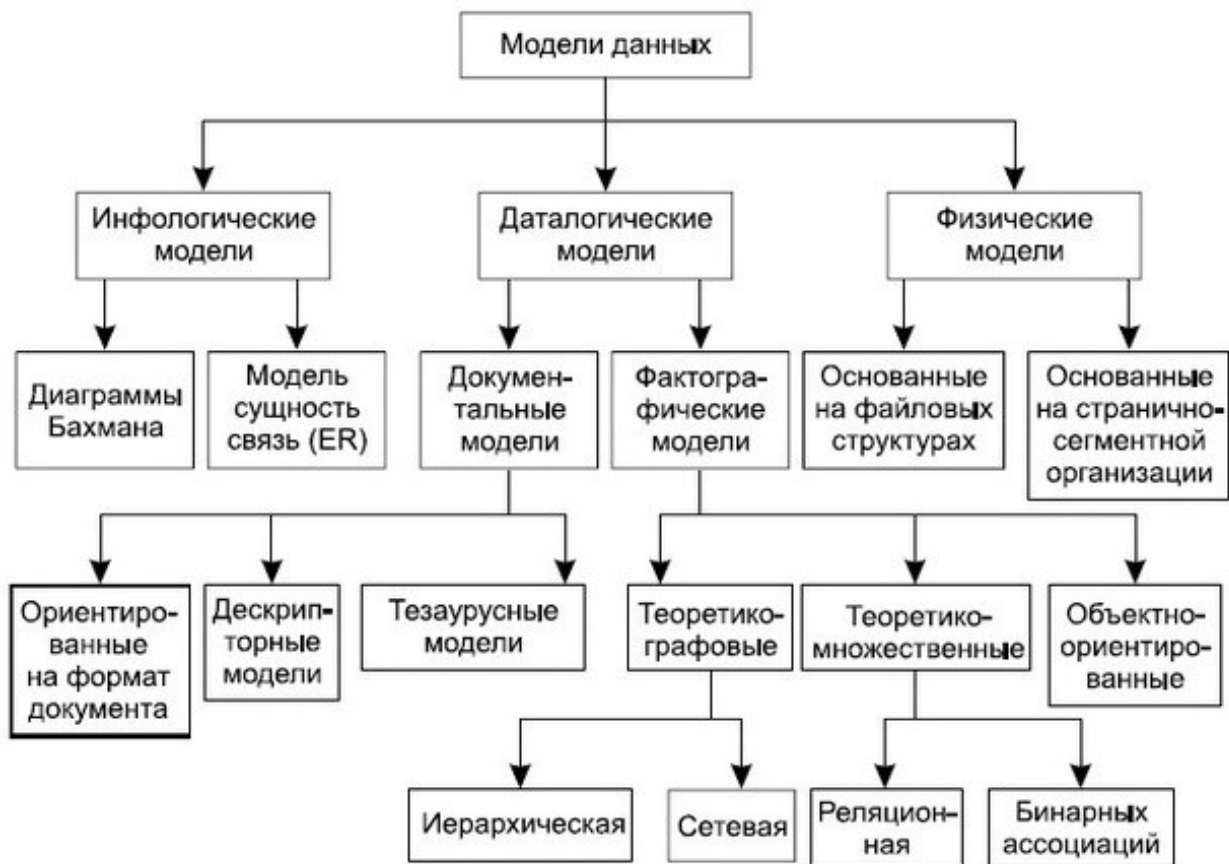
3.Физические - характеризуют распределение информационных ресурсов БД на конкретных физических носителях.

Документальные - соответствуют представлению слабоструктурированной информации.

Тезаурусные - основаны на принципах организации словарей. (в программных переводчиках)

Дескрипторные - В этих моделях каждому документу соответствует описание. Этот описатель (дескриптор) имеет жёсткий формат и является ссылкой на определённую в документах информацию

Теоретико-множественные модели основаны на теории множеств, опираются на свойства множеств и операции, которые производятся над множествами.



3 КОНЦЕПТУАЛЬНОЕ ПРЕДСТАВЛЕНИЕ ДАННЫХ

3.1 Концептуальное (семантическое) моделирование данных

Почему не сразу логическая (реляционная схема) данных?

1. Недостаточно средств для представления семантики предметной области
2. Плоские таблицы не всегда соответствуют предметной области со сложными объектами
3. Отсутствуют формальные средства представления зависимостей и ограничений

Концептуальная модель используется:

- Как инструмент проектирования и описания предметной области
- Для оценки ошибок стадии проектирования (например, в отношении полноты данных)
- Построение базовой документации по базе данных

Способы создания:

1. Ручной
2. Автоматизированный (как правило, до 3НФ без ограничений целостности)
3. Работа с базой данных на уровне семантической модели (ООБД)

Основные способы описания семантической модели:

1. ER-модель (entity – relation, модель сущность - связь) [различные нотации]
2. UML (модель классов в UML – Unified Modeling Language)

3.2 Сущность и ее свойства

3.2.1 Понятие сущности

Обычно проектирование базы данных начинается с построения концептуальной схемы; такая схема строится на основе знаний предметной области и требований к системе. Основным объектом рассмотрения в концептуальной схеме является сущность.

Сущность – это (в терминах концептуальной модели) формализованное описание некоторого концептуального объекта. Обычно один концептуальный объект представляется одной или несколькими связанными/зависимыми сущностями.

Каждая сущность состоит из атрибутов.

Фактически атрибут – это описатель ячейки данных; он представляет собой пару (имя: домен).

Атрибуты могут быть **обязательными** или **опциональными**.

- Обязательный атрибут кортежа не может быть незаполненным (т.е. не может содержать значения null), в отличие от опционального.

Сущность может быть изображена на схеме с разной степенью детализации:

Книга
Код ISBN
ББК
Автор
Название
Номер издания
Издательство
Год издания
Количество страниц

А: атрибуты представлены только именами.

Книга
Код ISBN: Number
ББК: String
Автор: String
Название: String
Номер издания: Number
Издательство: String
Год издания: Number
Количество страниц: Number

Б: атрибуты представлены именами и доменами

Книга
Код ISBN: Number
ББК: String (AK2.1)
Автор: String (AK1.1)
Название: String (AK1.2)
Номер издания: Number (AK1.3)
Издательство: String
Год издания: Number
Количество страниц: Number

В: добавлены знаки альтернативных ключей.

Прим: обязательность выполнения первичного ключа в СУБД - нет

3.2.2 Ключи

Сущность может иметь несколько ключей.

Ключ – это группа атрибутов из набора атрибутов сущности, уникально идентифицирующий экземпляр сущности.

Ключ предназначен как для возможности найти концептуальный объект в базе, так и для гарантии уникальности объектов.

Потенциальный ключ непременно должен удовлетворять трём следующим требованиям:

1. определённость – все атрибуты ключа должны быть обязательными (т.е. не могут содержать значения null);
2. уникальность – набор значений атрибутов ключа любого кортежа сущности должен быть уникальным в пределах всех кортежей сущности;
3. неизбыточность – никакое подмножество атрибутов ключа не должно обладать свойством уникальности.

Один из потенциальных ключей сущности, наиболее подходящий, объявляется первичным ключом. Остальные потенциальные ключи называются альтернативными.

Если среди ключей нет подходящего или если сущность вообще не имеет ключей, то добавляется абстрактный (суррогатный) ключ (обычно состоящий из одного положительного целочисленного атрибута) и он объявляется первичным.

3.3 Связи концептуальных объектов

Концептуальные объекты могут зависеть друг от друга. Их зависимости классифицируем четырьмя категориями:

1. иерархическая категоризация (наследование),
2. агрегирование (владение),
3. ссылка,
4. множественная ассоциация (связь много-ко-многим).

Рассмотрим каждую категорию подробно.

3.3.1 Наследование и иерархическая категоризация

Наследование используется для конкретизации более обобщённой сущности конкретными атрибутами.

- **Базовая сущность** – это та сущность, которую конкретизируют;
- **унаследованная сущность** – это сущность, которая содержит дополнительные атрибуты.

Правила кардинальности для наследования:

1. некоторому экземпляру из базовой сущности может соответствовать один кортеж из унаследованной сущности;
2. любому экземпляру из унаследованной сущности обязательно соответствует ровно один кортеж из базовой сущности;
3. мощность унаследованной сущности лежит в промежутке от нуля до мощности базовой сущности включительно.

Первичный ключ унаследованной сущности должен быть в точности таким же, как и первичный ключ базовой; кроме того, первичный ключ унаследованной сущности должен ссылаться на первичный ключ базовой.

Пример. Базовая сущность – Человек; унаследованная сущность – Студент.

Атрибуты сущности Человек:

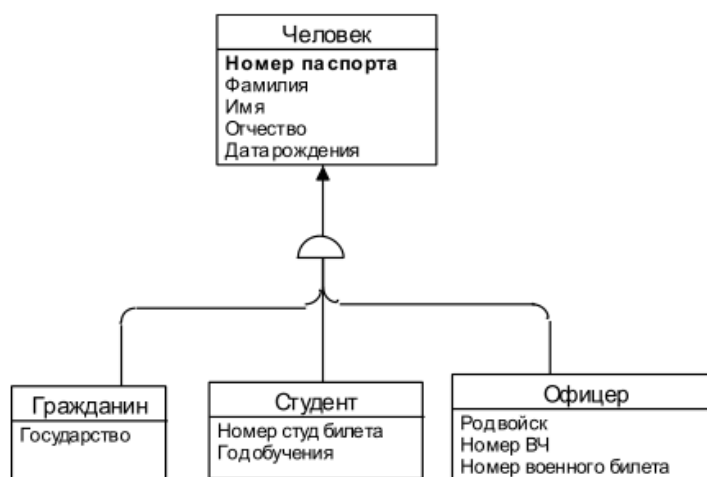
- ☐ номер паспорта (первичный ключ);
- ☐ фамилия;
- ☐ имя;
- ☐ отчество;
- ☐ дата рождения.

Атрибуты сущности Студент:

- ☐ номер паспорта (первичный ключ; ссылка на сущность Человек);
- ☐ номер студ. билета (альтернативный ключ);
- ☐ кафедра;
- ☐ год обучения.

В данном примере сущность первичный ключ сущности Студент полностью соответствует первичному ключу сущности Человек, и ссылается на него.

Иерархическая категоризация – это совокупность базовой сущности и нескольких от неё унаследованных.



Примечание. На концептуальных схемах мигрирующие атрибуты (в т.ч. унаследованные ключи) могут не изображаться¹.

Завершённая иерархическая категоризация – это вариант иерархической категоризации со специальными правилами кардинальности (которые добавляются к общим правилам кардинальности для наследования):

1. каждому экземпляру из базовой сущности всегда соответствует ровно один экземпляр из ровно одной унаследованной сущности;
2. сумма мощностей всех унаследованных сущностей в точности соответствует мощности базовой сущности.

С точки зрения реляционной алгебры категоризация является связью «один-к-одному».

3.3.2 Агрегирование

В иностранной литературе агрегирование часто называют master-detail.

Агрегирование (владение) – жёсткая ассоциация с кортежем одной сущности нескольких кортежей другой. В этом случае первая сущность называется родительской, а вторая – дочерней.

Агрегирование применяется в том случае, если кортеж дочерней сущности не может существовать без кортежа родительской (или это лишено смысла).

Первичный ключ дочерней сущности состоит из атрибутов первичного ключа родительской сущности плюс одного или нескольких дополнительных полей (*как правило одного*).

В качестве дополнительного поля ключа можно использовать какое-либо поле данных, уникальное в пределах кортежа родительской сущности (мастер-кортежа), либо абстрактный нумератор (enumerator). Важно, чтобы мигрирующие атрибуты помещались в начало ключа родительской сущности и в том же порядке, в котором они присутствуют в родительской.

Пример. Родительская сущность – Дом; дочерняя сущность – Квартира.

Атрибуты сущности Дом:

- ☐ улица (поле первичного ключа);
- ☐ номер дома (поле первичного ключа);
- ☐ этажность;
- ☐ общая площадь;
- ☐ архитектор.

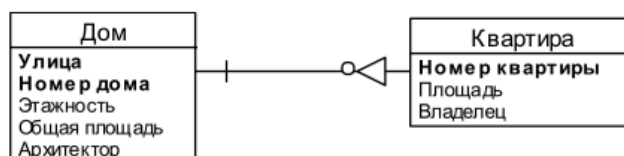
Атрибуты сущности Квартира:

- ☐ улица (поле первичного ключа, поле ссылки на сущность Дом);
- ☐ номер дома (поле первичного ключа, поле ссылки на сущность Дом);
- ☐ номер квартиры (поле первичного ключа);
- ☐ площадь квартиры;
- ☐ фамилия владельца.

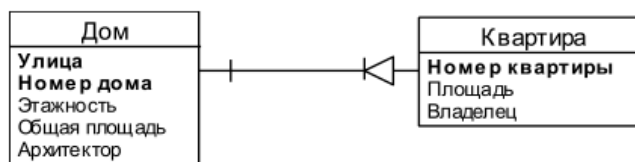
В данном примере первичный ключ сущности Квартира состоит из полей первичного ключа сущности Дом плюс дополнительного поля – номер квартиры.

Правила кардинальности:

1. некоторому экземпляру из родительской сущности могут соответствовать один или несколько экземпляров из дочерней сущности (или не одного);
2. каждому кортежу из дочерней сущности обязательно соответствует ровно один экземпляр из родительской сущности.



некоторому экземпляру из родительской сущности может соответствовать **РОВНО** один экземпляр из дочерней сущности;



С точки зрения реляционной алгебры агрегирование (владение) является связью «один-ко-многим».

3.3.3 Ссылка (ассоциация)

Ссылка – это такая связь, в которой кортеж одной сущности ссылается (указывает) на кортеж другой сущности. Сущность, на которую ссылаются, называется справочник (или словарь).

Для ссылки на кортеж справочника используется его первичный или альтернативный ключ (*В большинстве случаев используется первичный ключ. Некоторые СУБД вообще не позволяют ссылаться на альтернативные ключи*).

При ссылке ключ справочника мигрирует в тело (*но не в первичный ключ - прим*) ссылающейся сущности.

На самом деле, ключ справочника может мигрировать и в первичный (или альтернативный) ключ ссылающейся таблицы. Описанное выше правило имеет целью подчеркнуть, что атрибуты в большинстве случаев мигрируют в тело, а не в ключ ссылающейся таблицы.

Пример. Сущность Адрес ссылается на сущность Город.

Атрибуты сущности Адрес:

- ☐ персона, для которой хранится адрес (поле первичного ключа);
- ☐ номер адреса по порядку (поле первичного ключа);
- ☐ тип адреса;
- ☐ город 1 (ссылка на сущность Город);
- ☐ улица;
- ☐ номер дома;
- ☐ номер квартиры.

Атрибуты сущности Город:

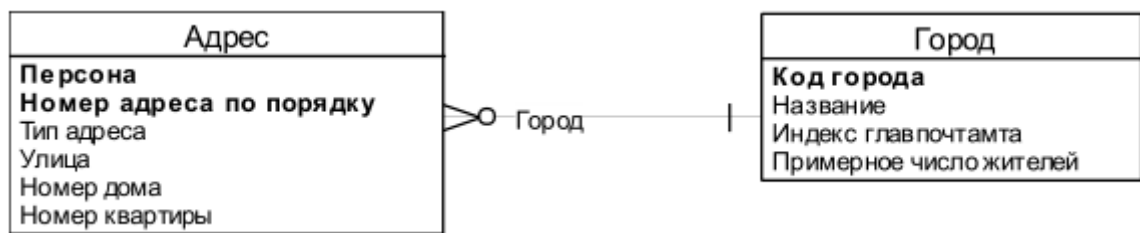
- ☐ код города (первичный ключ);
- ☐ название;
- ☐ индекс главпочтамта;
- ☐ примерное число жителей.

В данном примере атрибут «код города» в сущности Адрес – это ссылка на кортеж из сущности Город.

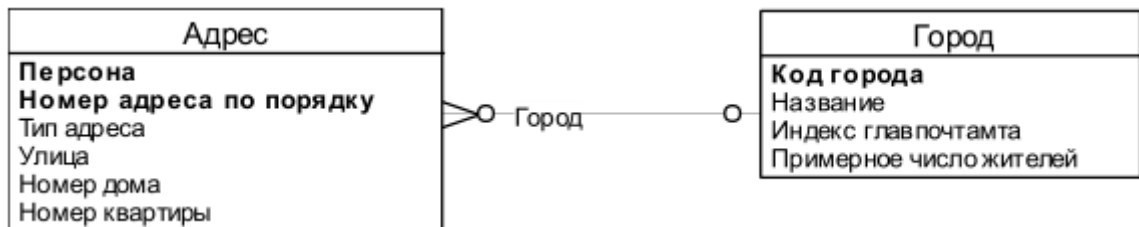
Ссылка может быть **обязательной** или **необязательной**: атрибуты обязательной ссылки всегда должны быть определены, т.е. не могут принимать значение null.

Ссылка может **состоять из одного поля** или **из нескольких** – в зависимости от числа полей в ключе справочника. В случае необязательной ссылки все поля ссылки в одном кортежа должны быть определены или пусты.

Обязательная ссылка изображается так:



Не обязательная:



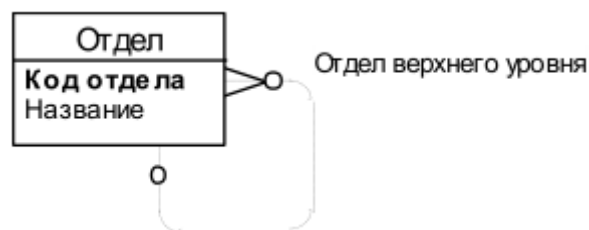
Если на схеме принято не отображать мигрирующие атрибуты, то имена атрибутов можно разместить на линии связи. Следует обратить внимание на то, что у сущности Адрес на самом деле 7 атрибутов, хотя явно показано только 6.

Правила кардинальности:

1. некоторому экземпляру из ссылающейся сущности может соответствовать один экземпляр из справочника;
2. некоторому экземпляру из справочника может соответствовать сколько угодно экземпляров из ссылающейся сущности.

С точки зрения реляционной алгебры ссылка является связью «многие-к-одному».

С помощью ссылки можно представить кортежи в виде дерева, если сделать ссылку на себя:

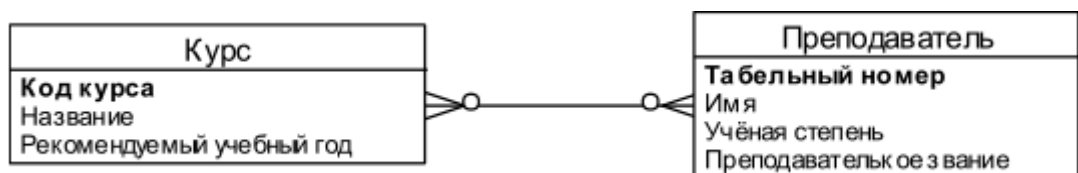


В данном случае сущность Отдел содержит три атрибута. Головной отдел – тот, для которого не задан атрибут «отдел верхнего уровня».

3.3.4 Множественная ассоциация

Множественная ассоциация – это случай, когда любой кортеж одной сущности может быть ассоциированным с любым количеством кортежей другой сущности, и наоборот.

На концептуальной схеме изображается так:



О том, как реализуется множественная ассоциация на практике, будет рассказано в главе, посвящённой физической схеме.

С точки зрения реляционной алгебры множественная ассоциация является связью «многие-ко-многим».

3.4 Порядок инфологического (семантического) проектирования БД

3.4.1 Разработка семантической/концептуальной модели

Порядок разработки концептуальной модели базы данных:

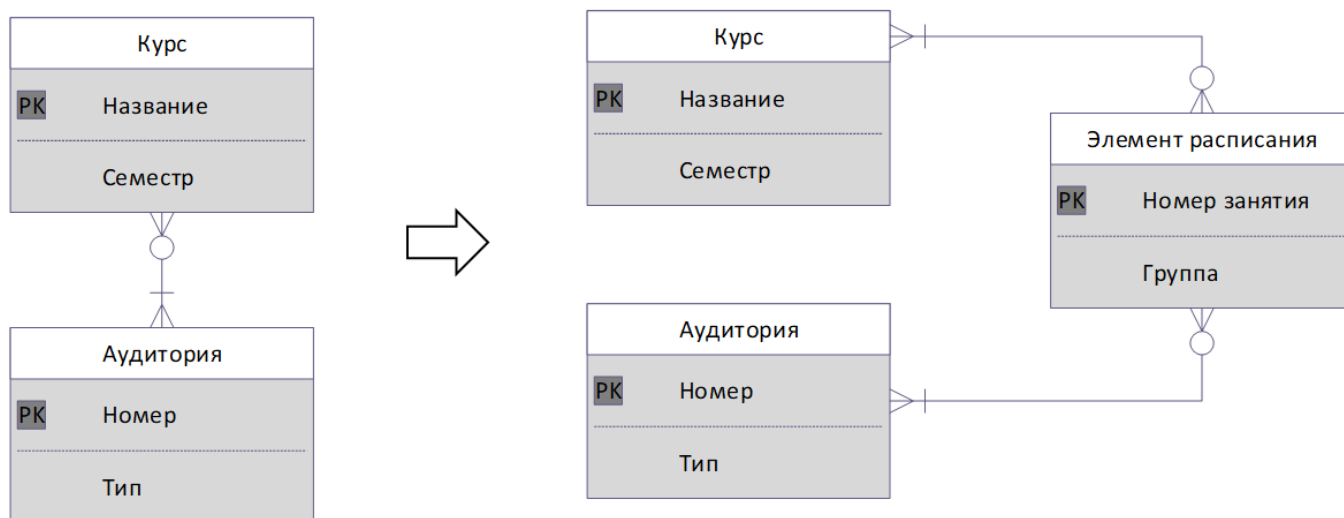
1. Анализ (знакомство) с предметной областью
2. Определение пользователей (групп пользователей) данных
3. Определение необходимых и достаточных сведений для каждого пользователя
4. Выделение хранимых сущностей (исходя из принципов полноты и достаточности).
5. Определение связей и их характеристик
6. Определение атрибутов
7. Проверка и корректировка модели (итерационно)
8. Определение потенциальных (первичных и альтернативных) ключей
9. Определение ограничений

Результат формализуется в виде ER – схемы, например, в приведенных выше нотациях.

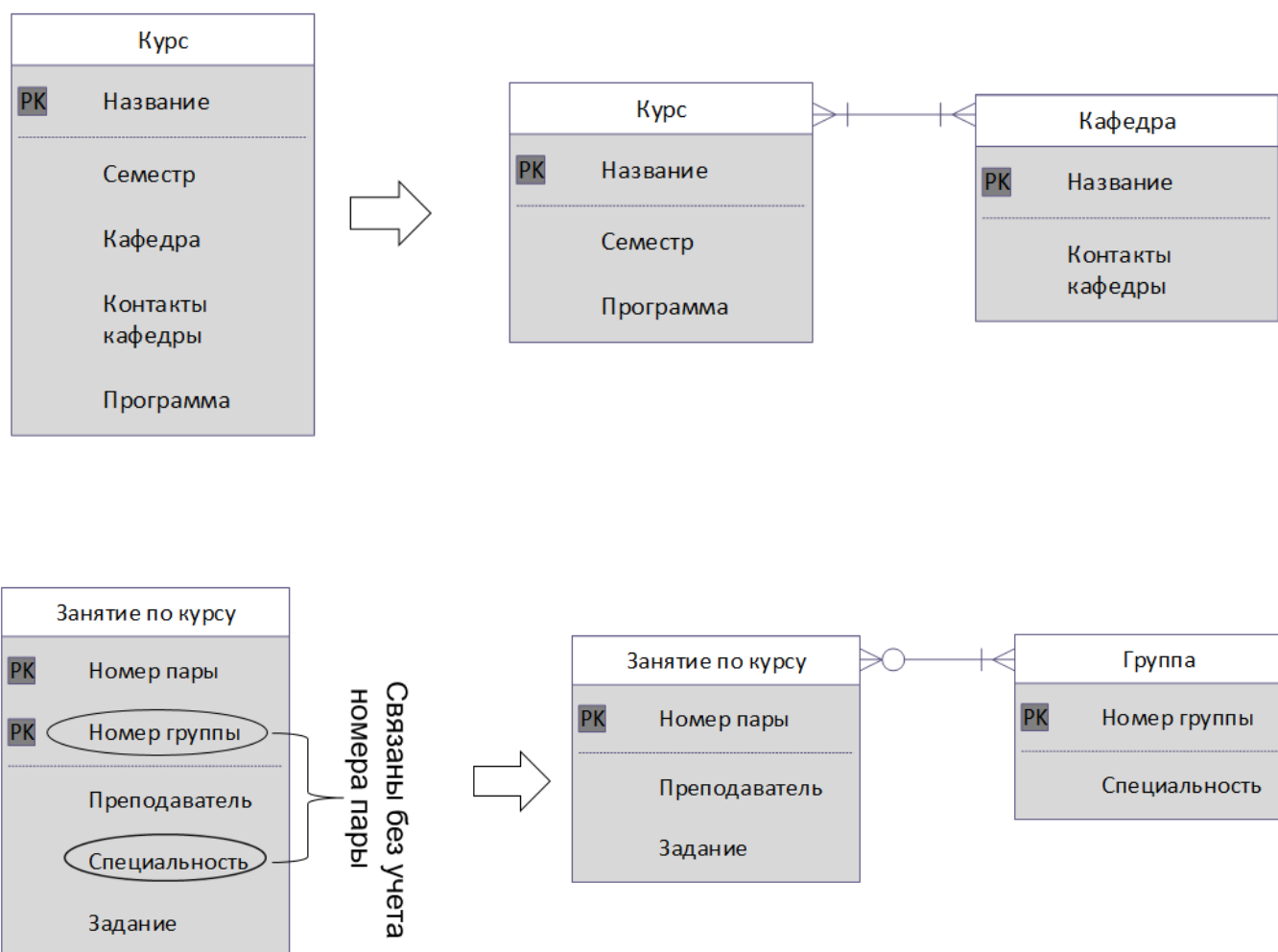
3.4.2 Верификация семантической/концептуальной модели

Выделение скрытых сущностей

- а. Выделение сущности если у нее есть отдельные, собственные свойства



- б. Выделение в отдельную сущность атрибутов, связанных друг с другом, но не напрямую зависящих полностью от ключевого атрибута



3.5 Описание ограничений целостности

Первая задача по безопасности данных – обеспечение целостности или нормального функционирования БД. Работоспособность БД должна наименьшим образом зависеть от квалификации пользователя, а для этого должны соблюдаться все ограничения целостности.

1. Описание на естественном языке
2. Описание на языке OCL (Object Constrains Language)

3.5.1 Виды целостности (обеспечения целостности) в СУБД

Структурная целостность, СУБД должна допускать работу только с однородными структурами данных (например, типа «реляционное отношение», при этом понятие «реляционного отношения» должно удовлетворять всем ограничениям, накладываемым на него в классической теории реляционной БД).

Языковая целостность, которая состоит в том, что язык манипулирования СУБД должен отвечать операциям модели данных. .

Ссылочная целостность (Declarative Referential Integrity, DRI), означает обеспечение связей, заявленных в модели данных. Ссылочная целостность обеспечивает поддержку непротиворечивого состояния БД в процессе модификации данных при выполнении операций добавления или удаления.

Кроме указанных ограничений целостности, которые в общем виде не определяют семантику БД, вводится понятие **семантической поддержки целостности**.

3.5.2 Понятие ограничения целостности (с точки зрения ссылок и семантики)

Ограничение целостности — это логическое выражение, связанное с некоторой базой данных, результатом вычисления которого всегда должно быть значение TRUE.

Подобное ограничение может рассматриваться как формальное выражение некоторого бизнес-правила.

В общем, ограничения целостности представляют собой ограничения, налагаемые на значения, которые разрешено принимать некоторой переменной, или комбинации переменных (в случае БД – переменных отношения).

Каждое ограничение представляет собой логическое выражение (импликацию) в котором участвует переменная. Т.е. это высказывание служит **предикатом** а переменная – **формальным параметром предиката**. При проверке ограничения вместо формального параметра подставляется фактический.

3.5.3 Когда проводить проверку

Исходя из этого проверка ограничения должна происходить ДО вставки данных в отношение. В современных СУБД возможны отложенные проверки для удобства работы.

В литературе чаще всего утверждается или просто предполагается, что единицей контроля целостности является транзакция и что часть проверок следует откладывать до конца транзакции (т.е. ко времени выполнения операции COMMIT – завершение транзакции).

3.5.4 Классификация ограничений целостности

■ Ограничение типа представляет собой не что иное, как определение множества значений, из которых состоит данный тип. Является спецификацией значений, из которых состоит рассматриваемый тип. (Год – целое от 0000 до 3000)

■ Ограничением атрибута называется ограничение на значения, которые разрешено принимать указанному атрибуту. Ограничением атрибута фактически является просто объявление, которое гласит, что указанный атрибут указанной переменной отношения имеет указанный тип (априорные ограничения). (Год рождения – Год).

■ Ограничением сущности называется ограничение на значения, которые разрешено принимать переменной соответствующей сущности концептуальной модели (или соответствующему элементу БД). (Год рождения - от 1900 до 2015)

■ Ограничением базы данных называется ограничение на значения, которые разрешено принимать указанной базе данных. То есть, на несколько сущностей. (Поставщики с кодом от 5000 до 6000 могут поставлять в сумме не более 10 видов деталей).

Подвид ограничения переменной/базы данных:

Ограничением перехода называется ограничение, регламентирующее допустимые переходы для определенной переменной (в частности, для конкретной переменной отношения или базы данных), которые она может выполнять, переходя от одного значения к другому.

Например, семейное положение любого лица может измениться со значения "никогда не состоял в браке" на значение "состоит в браке", но возврат к прежнему состоянию невозможен.